

ENUNCIADO DO TRABALHO T4

INTRODUÇÃO AO T4

O T4 compõe-se de 4 Projetos, cujo objetivo geral é lidar com um sistema computacional complexo (MIPS_S_withBRAMs) e desenvolver o hardware de um Periférico capaz de interagir com este. A abordagem incremental é

1. Projeto 1 – Repetir os passos de como simular o Hw do sistema MIPS_S_withBRAMs executando uma aplicação (Slides 7 a 19 acima), para dominar a estrutura deste
2. Projeto 2 – Desenvolver um software capaz de transferir uma quantidade grande de dados de um arquivo para a memória de um periférico e daí copiá-los para a memória de dados do sistema MIPS_S_withBRAMs, gerando a estrutura de memória a usar nos Projetos 3 e 4
3. Projeto 3 – Com a estrutura de memória montada no Projeto 2, simular o Sw (devidamente modificado) sendo executado pelo Hw do sistema, avaliando sua execução
4. Projeto 4 – Criar um Periférico e um módulo topo. O primeiro deve se plugar na interface do MIPS_S_withBRAMs, conter uma máquina de estados, uma memória local, e interfaces para dispositivos de entrada e saída. O Periférico deve permitir: (1) transferir dados da sua memória local para o sistema; (2) Permitir ao usuário externo verificar a transferência

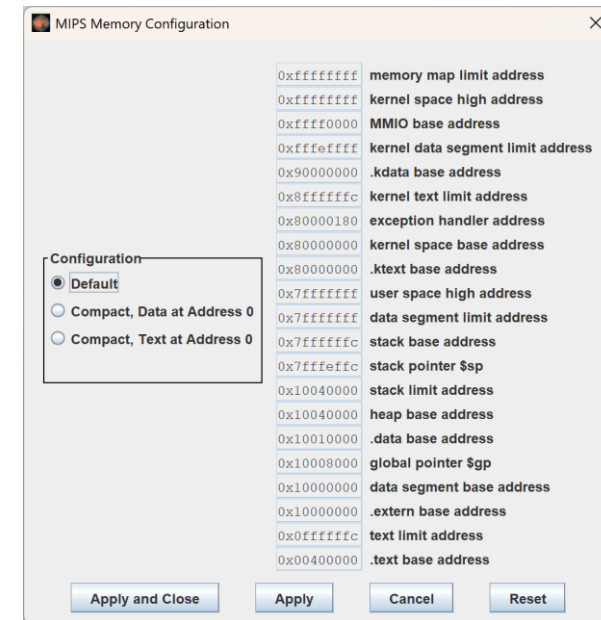
O MAPA DE MEMÓRIA DO MIPS_S A USAR

The MIPS_S Memory Map for the MIPS_S_withBRAMs system. This map is based on the Default Memory Model from the MARS Simulator Environment and is supported by MARS. The size of the memory pieces are not drawn proportionally.

0x00000000	Not Used (4MBytes, unpopulated)	
0x003FFFFF	Program Memory (User Data Segment, a 2-Kbyte BRAM)	MIPS_S Program Memory
0x00400000	Not Used (252MBytes-2Kbytes, unpopulated)	
0x004007FF		
0x00400800		
0x100FFFFF		
0x10010000	Data Memory (Data Segment, 16-Kbytes, 8 BRAMs)	MIPS_S Data Memory
0x10013FFF		
0x10014000	Data Memory (Data Segment, 16-Kbytes, 8 BRAMs)	Peripheral Data Memory
0x10017FFF		
0x10018000	Not Used (3,768GBytes-96Kbytes, unpopulated)	
0xFFFFEFFF		
0xFFFF0000	MMIO (64Kbytes, unpopulated)	Reserved For Peripheral Address Mapping
0xFFFFFFFF		

The stack pointer (\$sp) base address should be initially set to 0x10014000 (Empty Stack)

1. O MIPS_S, como o MIPS, usa endereços de 32 bits, cada um destes sendo o endereço de 1 byte
2. O MARS usa como modelo *default* um mapa de memória com diversos segmentos →→→→→
3. O MIPS_S usam apenas 3 destes segmentos: 1 para a Memória de Programa (2KBytes), 1 para a Memória de Dados (32Kbytes divididos entre o processador e periféricos e um segmento para mapeamento de periféricos
4. À esquerda se mostra a estrutura do mapa do MIPS_S - regiões de memória realmente utilizáveis são coloridas
5. *Unpopulated* significa que não existe memória real



PROJETO 1 (1)

1. Repitam todo o processo descrito nos slides anteriores para executar o programa exemplo *SomaVet.asm* no MIPS_S prototipável, usando simulação de hardware (e.g. com o ISIM)
 - Partam do programa em linguagem de montagem (*assembly language*, **.asm**), estudem e entendam ele
 - Simulem o mesmo, usando o simulador do MIPS, o MARS
 - Gerem os *dumps* dos segmentos de texto e de dados, conforme mostrado
 - Rodem o programa **le_mars** sobre os *dumps*, para gerar o arquivo **memory.vhd**

PROJETO 1 (2)

- **Atenção:** Nesta versão do sistema **MIPS_S_withBRAMs** usa-se uma Memória de Dados de 16Kbytes, composta por 2 módulos de memória entrelaçada com 4 BRAMs cada. O segundo módulo não é usado pelo *SomaVet.asm*, mas será útil mais tarde. A geração com *le_mars* usa os parâmetros opcionais **ni=1** e **nd=2**
- Gerem um projeto no ISE usando os arquivos VHDL do ambiente (como mostrado) e com o **memory.vhd** gerado
- Dominem neste projeto o processo de usar o ambiente de simulação do processador MIPS_S com blocos de memórias (BRAMs), o **MIPS_S_withBRAMs**

PROJETO 1 (3)

2. No material de apoio, há um documento de referência que ajuda para realizar este projeto, com mais detalhes que aqui
 - ✓ [5_5-Simulacao_MIPS_S_withBRAMs_V5.9.pdf](#)

PROJETO 2(1)

1. Escrevam um Sw em linguagem de montagem do MIPS capaz de transferir texto de uma região de memória para outra de mesmas dimensões, similar à função **strcpy** da linguagem C. Para tanto, assumam que
 - a. Tem-se disponível um ponteiro (ou seja, um endereço de memória) para a **área de memória fonte** dos dados a copiar e um ponteiro para o início da **área de memória destino** dos dados. Os dados terminam com um caractere NULL (0x00)
 - b. A **área de memória destino** tem espaço suficiente para receber os dados copiados
 - c. A **área de memória destino** está na Memória de Dados do MIPS_S (que possui 16384 bytes [16KB]: do endereço 0x10010000 até 0x10013FFF)
 - d. A **área de memória fonte** está em outro lugar da Memória de Dados do MIPS_S (tipicamente em um módulo disponível no arquivo **memory.vhd**, mapeada para endereços que o MARS pode usar. Estes devem ser compatíveis com uma região de memória a ser usada pelo periférico, futuramente conectado ao sistema MIPS_S_withBRAMs. **Use a região do mapa do MIPS a partir do endereço 0x10014000**
 - e. A interface de controle com o periférico **deve ser mapeada na região MMIO, a partir do endereço 0xFFFF000**

PROJETO 2(2)

Observação: Este arquivo tem de estar no mesmo diretório do executável do MARS

2. Validem o programa do item 1) no MARS

- Usem o arquivo fornecido (**Romeo&Juliet_Act2-Scene2.txt**) como entrada textual, empregando chamadas do sistema (system calls) para abrir e ler o arquivo para memória
- Usem a área de dados abaixo para o programa a desenvolver

```
.data                # The MARS data space starts at address 0x10010000
dest_text_addr:      .word    0
                    .space    9200    # This completes the reserved quasi-9Kbytes
                                   # space for the source string
after_dest:          .word    0      # After the string end, put a label,
                                   # to enable visualizing the end of the data segment window
                    .space    7144    # jump to align to address 0x10014000
filename:             .ascii "Romeo&Juliet_Act2-Scene2.txt" # filename at the end, to facilitate..
source_text_addr:    .word    0      # After the string end, put a label,
                                   # to enable visualizing the end of the data segment window
                    .space    9200    # This completes the reserved quasi-9Kbytes for the destination string
after_source:         .word    0      # After the string end, put a label,
                                   # to enable visualizing the end of the data segment window
```

- Desenvolvam um programa para realizar a função `strcpy`, mas que antes chame uma subrotina que carrega o texto na memória, a partir do ponteiro `source_text_addr` (que aponta para 0x10014000 no MARS)

PROJETO 3 (1)

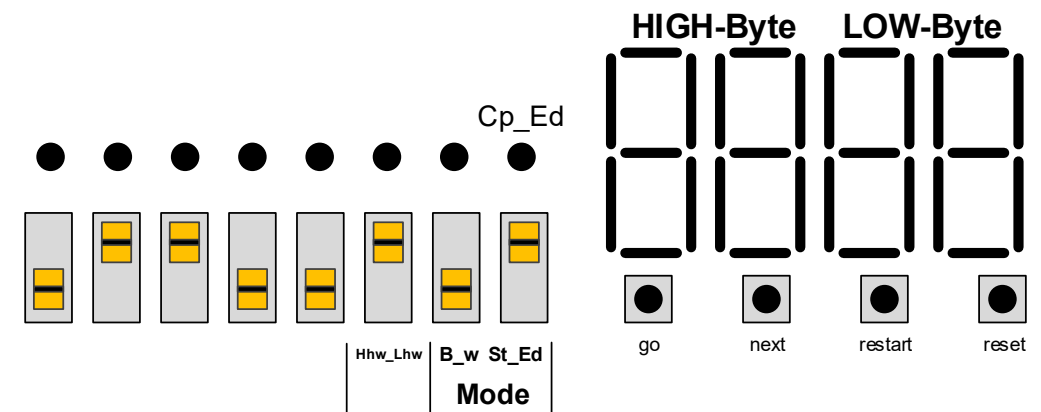
1. Depois de validar o programa no MARS, alterem ele e montem um ambiente de simulação do sistema MIPS_S_Sim, a exemplo do que foi feito para o programa exemplo **SomaVet.asm** com o sistema MIPS_S_withBRAMs. As alterações no programa são
 - a. Não usem a rotina de leitura do arquivo para a memória (a memória será pré-carregada de outra forma, pois o MIPS_S não consegue, ainda, processar arquivos...)
 - b. Alterem o final do programa, para que este termine com um loop eterno, do tipo
aqui: j aqui
Isto facilita a visualização do fim da simulação...
2. Desta vez, não usem os arquivos do ambiente MIPS_S_withBRAMS, mas o ambiente com a versão simulável do processador, conteúdos do diretório **2_MIPS_S_Sim**
3. Leiam o arquivo **README.txt** deste diretório, para entender como completar o testbench e simular seu programa no MIPS_S
4. Vejam no slide a seguir alguns cuidados de como preparar a simulação
5. Tendo dificuldades para fazer a simulação, consultem o professor...

PROJETO 3 (2)

1. O programa MARS na versão mais recente (V. 4.5) possui algumas características indesejáveis (limitações e/ou *bugs*) para realizar o *dump* de áreas de memória. Isto é particularmente sério no caso de áreas de memória grandes.
2. Como no Projeto 2 usa-se 32Kbytes de memória de dados, estas características/*bugs* atrapalham um pouco. O *dump* da área de texto (o programa) pode ser feito como mencionado antes, mas o *dump* da área de dados necessita de tratamento diferenciado
 - a. Primeiro, carregue no MARS o programa gerado para este projeto como descrito no slide intitulado **Projeto 3(1)**. Faça o *dump* da área de texto como explicado antes
 - b. Segundo, carregue no MARS o programa gerado no **Projeto 2**, execute-o e só então faça o *dump* da área de dados como explicado antes. Isto deve gerar um *dump* da área de dados completa (de 0x10010000 a 0x10016FFF)
 - c. Terceiro, edite o arquivo data.txt (ou o nome usado para o *dump*), eliminando os dados da cadeia nas primeiras 287 linhas deste (que contêm o texto já copiado). Usem para isto um editor como capacidade de seleção de qualquer conjunto de linhas e colunas de texto, como o Notepad++. Eu copiei os dados, sem incluir os endereços, das linhas 301-400 e sobrepus estes dados nas linhas 1-100 e 101-200. Depois copiei o quadro das linhas 301-387 e sobrepus estes dados nos dados das linhas 201-287. Outras soluções existem, claro. Salve o novo data.txt e continue a partir do item 3 do slide anterior (intitulado **Projeto3(1)**)...

PROJETO 4 – Especificação (1)

1. Acrescentar um periférico (hardware) ao sistema MIPS_S_withBRAMs. Este conterá lógica para realizar a transferência do texto longo do arquivo (**Romeo&Juliet_Act2-Scene2.txt**) sob comando de um usuário externo, devidamente carregado em uma memória física do periférico, para a memória de dados interna do sistema MIPS_S_withBRAMs. Como realizar isto?
 - a) A interface com o mundo externo deve ser aquela dada abaixo, que usa os 4 push-buttons, 3 chaves deslizantes, 1 led e o display da placa Nexys e onde se tem
 - reset – botão de inicialização geral do sistema (módulo topo)
 - go, next, restart – controle de visualização do texto copiado (**detalhes adiante**)
 - Cp_Ed – Led que avisa quando a cópia do texto foi concluída
 - Mode – duas chaves que controlam o modo de visualização – B_w escolhe ver byte a byte (SW1 em 0) ou word a word (SW1 em 1); St_Ed escolhe se visualização parte do início do texto (SW0 em 0) ou do fim do texto (SW0 em 1)
 - Hhw_Lhw – chave que controla o que se vê no display no modo word a word – SW2=0 indica que display mostra a meia palavra baixa, SW2=1 indica que a meia palavra alta é visualizada



PROJETO 4 – Especificação (2)

- b) Será fornecido um esqueleto do periférico a implementar (uma entidade e uma arquitetura parcial), contendo uma interface com o sistema MIPS_S_withBRAMS e uma memória devidamente preenchida com o texto e localizada em endereços mapeados iniciando no endereço 0x10014000 do MIPS_S
- c) A partir do esqueleto dado em b), gera-se uma máquina de estados que
 - i. Ao ser ressetada, aguarda ordem do usuário (botão 'go' apertado) para realizar a cópia do texto de sua memória para a do processador
 - ii. Avisa MIPS_S que 'go' foi apertado (posição de memória legível) e suspende a operação do MIPS_S para a transferência
 - iii. "Des-avisa" que 'go' foi apertado e realiza transferência, armazenando o tamanho do texto
 - iv. Ao final da cópia avisa fim ao MIPS_S (via posição de memória legível por ele) e ao usuário (Led CP_Ed acende); em seguida aguarda 'go' apertado e lê 'Mode' das chaves, setando seu valor e acendendo 'go' novamente para avisar ao MIPS_S
 - v. Des-suspende o MIPS_S. A partir daí aguarda sequência de comandos de visualização (MIPS_S enviada dados para o display e aguarda comandos 'next' ou 'restart' do usuário

PROJETO 4 – Especificação (3)

- d) Em paralelo com a máquina de estados do periférico, o MIPS_S executa um software (a ser desenvolvido). A estrutura geral deste compõe-se de quatro laços em que o processador interage com o periférico
 - i. Após ser inicializado o MIPS_S entra no **laço 1**, lendo a informação sobre o botão 'go' ter sido apertado ou não. Sai-se do laço ao detectar o apertar do botão
 - ii. Após sair do **laço 1**, o MIPS_S entra no **laço 2**, onde aguarda o fim da cópia do texto para a sua memória local pelo Periférico, que deve suspender o MIPS_S para isto
 - iii. Ao sair do **laço 2**, isto significa que a cópia foi realizada, e que é provável que o modo de visualização já tenha sido definido e o botão 'go' foi novamente apertado. O MIPS_S então executa o **laço 3**, onde lê o fato de 'go' ter sido apertado junto com o valor de 'mode'. Quando isto acontece, MIPS_S sai deste laço.
 - iv. Com o valor de 'Mode', o MIPS_S define o endereço de início do texto copiado, talvez lendo o 'text_size' do Periférico (se St_Ed='1'), lê primeiro dado (byte ou word) e envia para o Periférico, setando 'Show' para '1'
 - v. Em seguida, o MIPS_S entra no **laço 4**, onde aguarda o apertado de um dos botões 'next' ou 'restart' para re-enviar novo dado (a partir do início se 'restart' foi apertado, senão incrementado o ponteiro de leitura (de 1 em 1 ou de 4 em 4, dependendo se B_w está em '0' ou '1', respectivamente)

PROJETO 4 – DICAS (1)

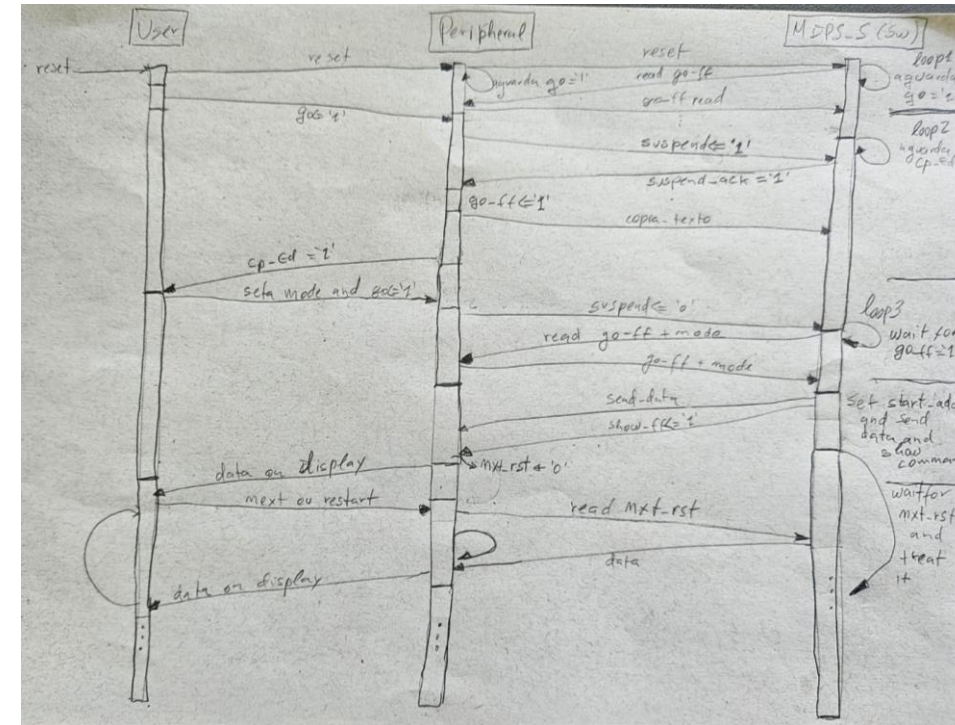
1. Usem a especificação para definir a estrutura da do Periférico e do Sw
2. A comunicação entre Sw (MIPS_S) e Hw (Periférico) deve usar técnicas de entrada e saída (E/S) mapeada em endereços (para o MIPS_S acessar dados do periférico) e acesso à memória do MIPS_S pelo periférico via suspensão do MIPS_S (para realizar a cópia do texto do Periférico para a memória do MIPS_S)
3. Da especificação do sistema, os endereços que o Sw deve usar para buscar ou enviar dados ao Periférico são 5 para leitura e 2 para escrita
 - i. Leitura
 - o valor do botão 'go' (1 bit)
 - o fato da cópia haver terminado 'Cp_Ed' (1 bit)
 - o mode de visualização 'Mode' (2 bits),
 - o fato de um dos botões next ou restart ter sido apertado 'nxt_rst' (2 bits)
 - o tamanho do dado copiado 'Text_size' (14 bits, no mínimo)
 - ii. Escrita
 - valor a ser mostrado no display, digamos 'Current_chunk' (32 ou 8 bits relevantes)
 - ordem do MIPS_S para habilitar que os dados sejam mostrados no display, 'Show' (1 bit)

PROJETO 4 – DICAS (2)

4. Cada um dos endereços corresponde a registradores no Periférico (7)
5. Sugere-se usar aqui endereços da região MMIO, 1 para cada valor, e.g. 0xFFFF0000 para 'Go', 0xFFFF0001 para 'CP_Ed' etc. É bom definir estes endereços na área de dados do Sw, de forma simbólica
6. Quando 'Mode' determina leitura palavra a palavra, lê-se 32 bits de texto por vez (4 caracteres ASCII), mas o display só pode mostrar 2 bytes de cada vez (2 caracteres ASCII em hexa). Assim, usa-se um multiplexador 2x(16 bits):1x(16bits) na saída, a chave 'Hhw/Lhw', dizendo o que vai no dspl_drv
7. Quando se usa leitura palavra a palavra do texto a partir do fim do texto ('Mode'=11) o Sw precisa tratar um possível caso especial na primeira leitura, pois a instrução lw do MIPS apenas pode ler de endereços múltiplos de 4. **Cuidado!**

PROJETO 4 – DICAS (2)

8. A estrutura dos dados nas memórias de dados do MIPS_S e do Periférico deve ser aquela obtida no Projeto 3 (ver item 2. do slide 29) para obter o arquivo data.txt (*dump* do **Data Segment**). Com o Sw do MIPS_S desenvolvido, o prog.txt deve ser apenas o *dump* do do **Text Segment** deste
9. Para ajudar a elaboração do sistema, segue ao lado um esboço grosseiro de um Diagrama de Sequência para o sistema, mostrando as ações do Usuário, do Periférico e do MIPS_S (Sw) →→→→→→→→→→→



Bad News → A quantidade de BRAMs exigida neste projeto (17) não permite que o FPGA da Nexys 1 (XC3S200) seja usado, pois ele só possui 12 BRAMs – Para prototipar o sistema, compartilhem as Nexys 2 com grupos que têm ela...

RESUMO DO TRABALHO T4

- O **Trabalho T4** a ser entregue consiste em um arquivo compactado (.zip) contendo
 - ✓ Um relatório em PDF descrevendo os Projetos 2-4 (2 pontos)
 - ✓ Códigos fonte *assembly* gerados, comentados (Projeto 2) (2 pontos)
 - ✓ Diretório com os VHDLs (Projetos 3 e 4), arquivos .ucf e .bit (Projeto 4) (3 pontos)
 - ✓ O projeto completo ISE/Vivado do protótipo gerado (Projeto 4) (3 pontos)
 - ✓ **DEVE-SE** mostrar a operação do projeto na plataforma de prototipação ao professor em alguma ocasião, ou enviar um vídeo onde a operação correta é demonstrada